

File Upload & Download in Playwright

Theory + Hands-on Coding

File Handling in Playwright

Playwright provides direct APIs for uploading files and event-based handling for file chooser and downloads.

Learning Goal

Understand how to upload a file, handle a file chooser, download a file, and save the downloaded file with either its original or a custom name.

Topics

File Upload → Direct `setInputFiles()` → File Chooser Event → File Download → `saveAs()` → `suggestedFilename()`

File Upload

Direct file upload using setInputFiles()

What is file upload?

Uploading means selecting a file from the local machine and providing it to an application's file input control.

Direct approach

When the file input element is directly accessible, Playwright can set the file using locator.setInputFiles().

No manual file dialog

You do not need to interact with the operating system file picker manually when using setInputFiles().

Example from the training code

```
await page.goto("https://demoqa.com/upload-download");

await page.getByRole("button", {name: "Choose File"})
.setInputFiles(path.join(__dirname, "../../Data/Logo.png"));
```

File Upload

Understanding setInputFiles()

Locate the upload control

Identify the file input element using a reliable locator.

setInputFiles()

Pass the local file path to setInputFiles().

path.join()

path.join() creates a platform-independent path from the current test location to the file.

Syntax

```
await locator.setInputFiles(  
  path.join(__dirname, "../../Data/Logo.png")  
);
```

Important

The file must exist at the path supplied to setInputFiles().

File Upload

When the file input is triggered through a file chooser

Why use the file chooser event?

Some applications expose a button or custom UI that opens the native file chooser instead of giving you a directly usable file input locator.

Step 1

Register `page.waitForEvent("filechooser")` before clicking the upload control.

Step 2

Click the element that opens the file chooser.

Step 3

Await the `FileChooser` reference.

Step 4

Use `setFiles()` on that `FileChooser` reference.

Training example

```
await page.goto(
  "https://blazorise.com/docs/components/file-picker"
);

const fileUploadPromise =
  page.waitForEvent("filechooser");

await page.locator("//span[text()='Choose files']")
  .first()
  .click();

const fileUploadRef = await fileUploadPromise;

await fileUploadRef.setFiles(
  path.join(__dirname, "../../Data/TestleafLogo.png")
);
```

File Upload

File Chooser event flow

STEP 1

Register the event before the action

STEP 2

Click the upload button

STEP 3

Wait for the FileChooser object

STEP 4

Call setFiles()

STEP 5

The application receives the selected file

Key point

The event must be registered before the click because the click triggers the file chooser event.

File Download

Handling downloads with `waitForEvent()`

What is file download handling?

A download starts when the application provides a downloadable file and the user triggers the download action.

Register the event

Use `page.waitForEvent("download")` before clicking the download control.

Click the download button

Perform the action that starts the download.

Get the Download object

Await the download promise and store the returned reference.

Training example

```
await page.goto(
  "https://demoqa.com/upload-download"
);

const fileDownloadPromise =
  page.waitForEvent("download");

await page.getByRole("button", {name:"Download"})
  .click();

const fileDownload =
  await fileDownloadPromise;
```

File Download

Saving the downloaded file

Original file name

`download.suggestedFilename()` returns the filename suggested by the downloaded resource.

saveAs()

Use `saveAs()` to save the downloaded file to a location of your choice.

Custom file name

You can provide your own filename instead of using `suggestedFilename()`.

Training example

```
await fileDownload.saveAs(
  path.join(
    __dirname,
    `../../download/${fileDownload.suggestedFilename()}`
  )
);

await fileDownload.saveAs(
  path.join(
    __dirname,
    `../../download/TestleafLogo.png`
  )
);
```

File Download

suggestedFilename() vs custom filename

Original / suggested name	<code>fileDownload.suggestedFilename()</code>	→	Keeps the filename suggested by the download
Custom name	<code>TestleafLogo.png</code>	→	Saves the download using the name you provide
Destination	<code>.././download/</code>	→	Controls where the file is stored

Upload vs Download

Quick comparison

Upload	<code>locator.setInputFiles(filePath)</code>	→	Directly provides the local file to the file input
Upload via chooser	<code>page.waitForEvent("filechooser") + setFiles()</code>	→	Handles custom upload controls
Download	<code>page.waitForEvent("download")</code>	→	Waits for the download event
Save download	<code>download.saveAs(path)</code>	→	Stores the downloaded file at the required location

Complete Upload Example

DemoQA file upload

Complete Playwright code

```
import { test } from "@playwright/test";
import path from "path";

test("Test to verify file upload", async ({ page }) => {

  await page.goto(
    "https://demoqa.com/upload-download"
  );

  await page.getByRole(
    "button",
    { name: "Choose File" }
  ).setInputFiles(
    path.join(
      __dirname,
      "../../Data/Logo.png"
    )
  );

  await page.waitForTimeout(7000);
});
```

Observation

The training code uses setInputFiles() directly and then waits for 7 seconds.

Complete File Chooser Example

Blazorise file picker

Complete Playwright code

```
import { test } from "@playwright/test";
import path from "path";

test("Test to verify file upload", async ({ page }) => {

  await page.goto(
    "https://blazorise.com/docs/components/file-picker"
  );

  const fileUploadPromise =
    page.waitForEvent("filechooser");

  await page.locator(
    "//span[text()='Choose files']"
  ).first().click();

  const fileUploadRef =
    await fileUploadPromise;

  await fileUploadRef.setFiles(
    path.join(
      __dirname,
      "../../Data/TestleafLogo.png"
    )
  );

  await page.waitForTimeout(5000);
});
```

Complete Download Example

DemoQA file download

Complete Playwright code

```
import { test } from "@playwright/test";
import path from "path";

test("Test to verify file download", async ({ page }) => {

  await page.goto(
    "https://demoqa.com/upload-download"
  );

  const fileDownloadPromise =
    page.waitForEvent("download");

  await page.getByRole(
    "button",
    { name: "Download" }
  ).click();

  const fileDownload =
    await fileDownloadPromise;

  await fileDownload.saveAs(
    path.join(
      __dirname,
      `../../download/${fileDownload.suggestedFilename()}`
    )
  );

  await fileDownload.saveAs(
    path.join(
      __dirname,
      `../../download/TestleafLogo.png`
    )
  );

  await page.waitForTimeout(7000);
});
```

Observation

The example demonstrates both the suggested filename and a customized filename.

Best Practice Flow

File handling sequence

STEP UPLOAD - Direct

Locate file input → `setInputFiles(filePath)`

STEP UPLOAD - Chooser

`waitForEvent("filechooser")` → click → await chooser → `setFiles(filePath)`

STEP DOWNLOAD

`waitForEvent("download")` → click → await download → `saveAs(path)`

STEP PATH

Use `path.join(__dirname, ...)` to build the required file path

STEP VERIFY

Where needed, verify that the uploaded/downloaded file is present at the expected location

Summary

File Handling in Playwright

01

Use `setInputFiles()` when the file input can be accessed directly.

02

Use the `filechooser` event when the application opens a file chooser through a custom control.

03

Register `filechooser` before the action that opens it.

04

Register `download` before clicking the download control.

05

Use `download.saveAs()` to save a downloaded file.

06

Use `suggestedFilename()` when you want the filename suggested by the download.

07

Use a custom filename when you want your own destination name.

Mind Map

Playwright File Handling

File Handling

Upload + Download

Upload

`setInputFiles()`

Custom Upload

`waitForEvent("filechooser") → setFiles()`

Download

`waitForEvent("download")`

Save

`download.saveAs(path)`

Original Name

`download.suggestedFilename()`

Custom Name

Provide your own filename

Path

`path.join(__dirname, ...)`